

## CHAPTER 2

# The Order You Cannot See

## Order Lifecycle and State Machines

**Prerequisites:** Chapter 1, From Exchange Packet to Trade — Anatomy of an Electronic Trading System **Focus:** your order state is a *belief* about the exchange's state, not the state itself — and the pending states exist to make the gap between them visible

### The cancel that was too late

A firm has a resting buy order on Nasdaq: 500 shares of NVDA at \$178.40, sitting in the book, waiting.

News breaks. The strategy wants out, so the gateway sends a cancel.

While that cancel is travelling down the wire, someone hits the order. It fills. The exchange's matching engine — which has not yet seen the cancel — does what it is supposed to do and executes the trade.

For a few hundred microseconds:

- The **exchange** believes: order filled, 500 shares traded.
- The **firm** believes: order cancelled, no position, no exposure.

Both are acting on those beliefs. One of them is wrong, and it is not the exchange.

Now: the firm's system is about to receive two messages, in some order, and it does not control which. A fill for 500 shares. And a rejection of the cancel, because you cannot cancel an order that no longer exists. **Neither message is an error.** Both describe a completely normal outcome, and a system that treats either as an exception has a bug that will surface on a busy day.

### Where you will actually meet this

Every order gateway, execution management system, and pre-trade risk engine is built around this state machine. If you work anywhere near execution you will read it, extend it, or debug it.

"What happens if you send a cancel and it fills first?" is close to a mandatory interview question for execution roles. It is asked because the answer sorts people cleanly: engineers who have sent live orders answer it as routine, and engineers who have only read about trading treat it as a puzzle or an error case.

### The mental model

Here is the idea the whole chapter rests on.

**The exchange's view of your order is authoritative. You cannot see it.**

What you have is a local model — a belief — updated by messages that arrive after a network delay. Your belief lags reality by roughly one network trip, and during that window it can be wrong in ways you cannot detect from the inside.

That reframes what the state machine is for. It is not bookkeeping. It is a structured representation of *what you currently believe and how much you trust it*.

Which is why the states that matter most are the ones that mean **"I do not know yet."**

### Part 1 — The states, and why the pending ones exist

A simplified lifecycle. Real venues expose more, but these carry the ideas: